

EEC172 Final Project Report

Andre Vojtenyi, Hsien-I Tsou



Fig. 1. State Machine Diagram

I. DESCRIPTION

A simple game of Pong played with two different controllers. One of the paddles is controlled by the accelerometer on the CC3200, and the second one is controlled by the IR receiver and remote combo. After the game is over, the winning player is prompted to enter a three-letter name using the same algorithm as the one we wrote in Lab 3. AWS IoT is then used to send the name to a Google Sheets, which tracks and ranks the number of wins per name, which is then supposed to be displayed as a leaderboard.

II. DESIGN

The program begins by setting up an LED display with two paddles, a dashed midline, a ball at the middle of the

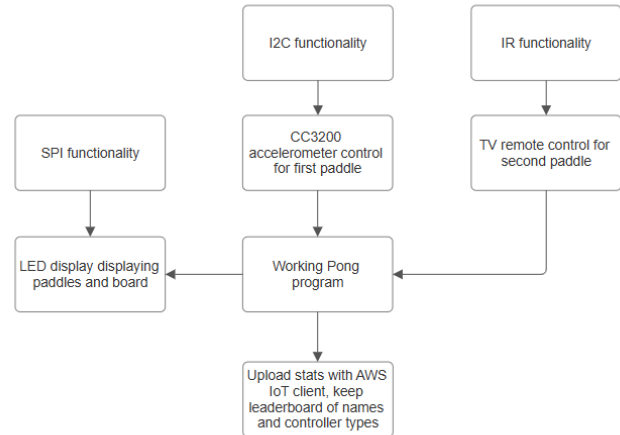


Fig. 2. Architectural Block Diagram

display, and printing the 0 to 0 scoreline at the top of the display. The game then runs with the following logic; the ball is sent towards the left side paddle with a set x velocity and a randomized y velocity. If the ball comes into contact with either the top or bottom bounds of the board, its y velocity is flipped. If it comes into contact with either of the paddles, its x velocity is flipped and slightly increased (it increases by a specified factor until it maxes out at a velocity cap), and its y velocity is slightly altered based of the paddles velocity at the time of contact. If the ball comes into contact with either the left or right side of the display, a point is scored. Player 1's paddle is controlled with the accelerometer on the CC3200, and player 2's paddle is controlled by the up and down buttons (the ones around the OK button) on the remote. Once a point is scored, the board resets. If the target score hasn't been reached, it runs another round, incrementing the scoring player's point, and sending the ball towards the loser of the last rally. The rounds then continue until one of the players reaches the target score. Once it detects a win, the program continues to its next part, printing out the player that won (player 1 or player 2) prompting for a three-letter name associated with the winner. The winner types their name with the remote utilizing a slightly modified algorithm from Lab 3, cycling through a button's associated letters with repeated presses of the same button, deleting mistakes with the Delete button, and confirming their input with the Enter button. Once confirmed, the program then converts the input into a message readable by the AWS rule we set up, sends it over to a selected google account, and then updates the leaderboard by either incrementing the name's number of wins if it already exists on the leaderboard, or adding as a new name with 1 win.

The four main components (aside from the program and board themselves) of the project are the I2C functionality, SPI functionality, IR functionality, and web service functionality. The SPI functionality ensures that the game is being properly displayed on the Adafruit. The I2C functionality ensures that the program can access the x tilt of the board. The IR functionality ensures that the board is able to read and use the signals outputted by the remote. The web service functionality ensures that the board is able to communicate the result and winner of each game with the leaderboard. During the first part of the program (while the game is being played), the program concurrently needs SPI functionality (to display the game on the board), I2C functionality (to control player 1's paddle, and IR functionality (to control player 2's paddle with the up and down buttons). During the second part of the program (when determining the name of the winner), the program concurrently needs SPI functionality (to display the current name as it's being written), and IR functionality (to be able to write the name using the remote). During the last part of the program (storing the result of the game), the program needs AWS functionality to communicate with the device shadow and send the necessary information with the shadow's rule.

III. IMPLEMENTATION

I2C: The I2C interface is used to communicate with a 3-axis accelerometer that controlled Player 1's paddle. Since the accelerometer is physically connected to the CC3200 using standard I2C lines (SCL and SDA), dev pins 1 and 2 on the board couldn't be used for anything else. On the software side, the CC3200 wrote to the accelerometer's internal register to request X-axis tilt data, then read back a signed byte representing the tilt angle. This value's translated into paddle velocity, with thresholds applied to ignore minor tilt fluctuations, which prevents jitter when the device is level. This process runs continuously during gameplay, enabling real-time control.

SPI: The wiring for the Adafruit LED is CC3200 dev pin 18 into the LED's reset, dev pin 15 into OC, dev pin 5 (the SPI clock) into the clock, dev pin 7 (SPI out) into SI (signal in), dev pin 21 into DC, and the CC3200's 3.3 V signal and GND into the LEDs respective ports. The LED serves as the game's graphical output, reflecting the paddles' positions, ball's position, and score increments during the game, and prompting the winner for their name. When the ball or paddle moves, its previous location is erased by redrawing the background, and then its new location is displayed. The functions used (drawChar, fillRect, and fillCircle) are supplied by Adafruit's function library.

IR: The wiring for the IR receiver is the boards 3.3 V pin running through a 100 ohm resistor into the receiver's Vs leg, the boards 3.3 V pin running through a 100 microfarad capacitor into the receiver's ground leg, the boards ground pin running into the receiver's ground leg, and the receiver's output leg running into the CC3200's dev pin 63. The IR receiver is used to allow Player 2 to control the game using the standard IR remote by detecting the incoming signal, which is converted in the software using a timing-based decoding routine. This reconstructs each IR transmission as a decoded value (which

it does by measuring the intervals between the falling edges of the signal being passed into the board, converting those intervals into either a 0 or a 1, and then converting that into an unsigned long), each of which corresponds to a specific remote button. While the game is being played, the main loop checks the value of the current decoded signal and reacts accordingly. The signal corresponding to the UP button on the remote makes player 2's paddle go up, and the signal corresponding to the DOWN button makes their paddle go down. Any other signal, or a 0, which corresponds to no button being pressed, is ignored. In the later part of the program, a multi-tap text entry system is also implemented to allow letter selection using the remote. While waiting for an input, a timer interrupt is used to detect when a button is pressed. Each time a button is pressed, a timer is started, and if the same button is pressed again within that time frame, the character cycles to the next character corresponding to it. If the timer expires without the same input, or if a new input is received at any time, the program sets that letter and moves on to the next character slot. Additionally, pressing the DELETE button removes the last entered letter, and pressing the ENTER button once all three letters have been chosen submits the name. Additionally, at the same time that the characters are being displayed on the board, they're also being saved in a buffer to allow the program to send them using HTTP POST later.

AWS: Once the game ends, the program connects to the internet via Wi-Fi and sends the winner's name to the device's shadow cloud server using a secure HTTP POST request. The CC3200 tries to establish a Wi-Fi connection using the SimpleLink network stack. Once it connects, the device constructs an HTTPS POST request containing the player's name and the necessary headers. This data is then processed by an AWS rule and converted into data that is usable by the script we've written to make the leaderboard.

IV. CHALLENGES

The biggest challenge we faced was mapping the pins. Since we used a lot of the same GPIO pins between labs, a lot of the pin configurations for different hardware components we had used overlapped each other, and required us to make a new pin configuration. The issue wasn't too difficult to solve, but it did take a bit of time to find a mapping in which none of the GPIO output/inputs overlapped with one another or with pins with different functionalities. We were able to easily overcome this issue with a bit of research reading over the board documentation, and then just troubleshooting the configs. Another major challenge we ran into was during the programming of the pong game, where once the ball reached a certain velocity, it would just go through the paddle instead of bouncing off. To fix this, we added a collision handler that would predict the ball's next position, and if the paddle intersected the line between the ball's calculated next position and its current position, it counted as a collision, and placed the ball right in front of the paddle, as well as inverted it's x velocity.

V. FUTURE WORK

We'd like to add more complexity to the web service part of the project. Instead of just tracking wins, we'd want to track a handful of additional stats, such as head-to-head matchups, win rates, and organizing games by the type of controller used (accelerometer or remote). We were also thinking about adding more unique features to the game itself, possibly adding power-ups that change the way the game is played, or adding functionality to make the paddles move in 2 dimensions instead of just 1.

VI. BILL OF MATERIALS

CC3200 Launchpad, various wires, ADAfruit OLED display, IR receiver, 100 ohm resistor, 100 microfarad capacitor, AT&T tv remote. All of these, apart from the wires, were handed out during the lab portion of the class. Male-to-female wires are necessary to connect the Launchpad's pins to a breadboard, and male-to-male pins can be added to make the circuit cleaner. Both of these can be bought in bulk for prices ranging from 5 to 10 dollars.